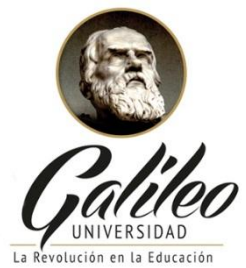


Universidad Galileo	Fecha 18/09/2026
Facultad: FISSIC	Alumno: Pablo Urbina
Curso: Microprocesadores	Carnet: 24001508
Sección: A	Hora de Laboratorio: 1:00 PM
Auxiliar:	Día de Laboratorio: Lunes
Brandon Eduardo Mendoza Espinoza	



Laboratorio # 5 Getting Started with Nucleo-G431KB

Objetivos:

Aprender el procedimiento necesario para desarrollar una aplicación de sistemas embebidos en una tarjeta Nucleo-G431KB por medio de Visual Studio en el lenguaje C.

Resumen:

Utilizamos los archivos que se nos dieron en el material para ejecutar un programa simple en la tarjeta el cual hace que el LED de la tarjeta parpadee rápidamente en dependencia del valor de una variable llamada **FREQ** la cual define la velocidad de frecuencia en Hz. También se analizaron las variables ticks y data, viendo como cambiaban sus valores incluyendo realizar una copia a la variable data para analizarla de mejor manera.

Teoría:

Un microcontrolador es un circuito integrado programable que integra una unidad central de procesamiento (CPU), memorias (Flash y RAM) y diversos periféricos de entrada y salida en un solo chip. La serie STM32G4, basada en el núcleo ARM Cortex-M4 de 32 bits, destaca por su alto rendimiento en procesamiento digital y su eficiencia en el manejo de interrupciones para aplicaciones en tiempo real. Incluye un temporizador decreciente de 24 bits integrado directamente en el núcleo, conocido como **SysTick**, el cual tiene como función principal generar interrupciones periódicas a intervalos fijos a partir del reloj central del sistema, su uso evita recurrir a bucles de espera pasiva, optimizando la ejecución del procesador.

Los puertos GPIO (Entrada/Salida de propósito general) permiten la interacción física con componentes externos, como LEDs. La conmutación del pin de salida digital dentro de una rutina de servicio de interrupción como **SysTick_Handler**, permite sincronizar eventos periódicos como el destello de un LED a una frecuencia configurada.

Datos Prácticos:

13. En 5 Hz:

https://drive.google.com/file/d/1IP-hf8QRFKBoT2t_Gh1HmtDCkxeV7u9l/view?usp=sharing

14. En 1 Hz:

<https://drive.google.com/file/d/1E7t3XKyMEfEa9v6xVG8NOFGWGDtReL9a/view?usp=sharing>

15. En 10 Hz:

<https://drive.google.com/file/d/1zPwNuus4HYfslPKTucYdKpZ-6aR19Ncn/view?usp=sharing>

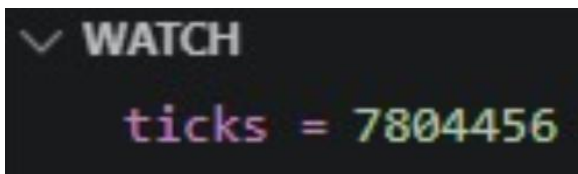
En 20 Hz:

<https://drive.google.com/file/d/1vmRUsgLYLVSSZO2DVR5okjhNLcYR3pG5/view?usp=sharing>

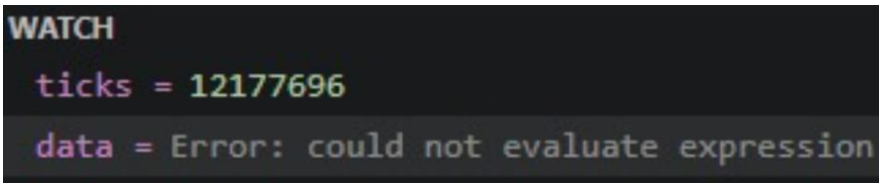
En 50 Hz:

https://drive.google.com/file/d/1lgKPW2Ns4cSsfr_U80KJgRaRN5CZ2CDX/view?usp=sharing

18.



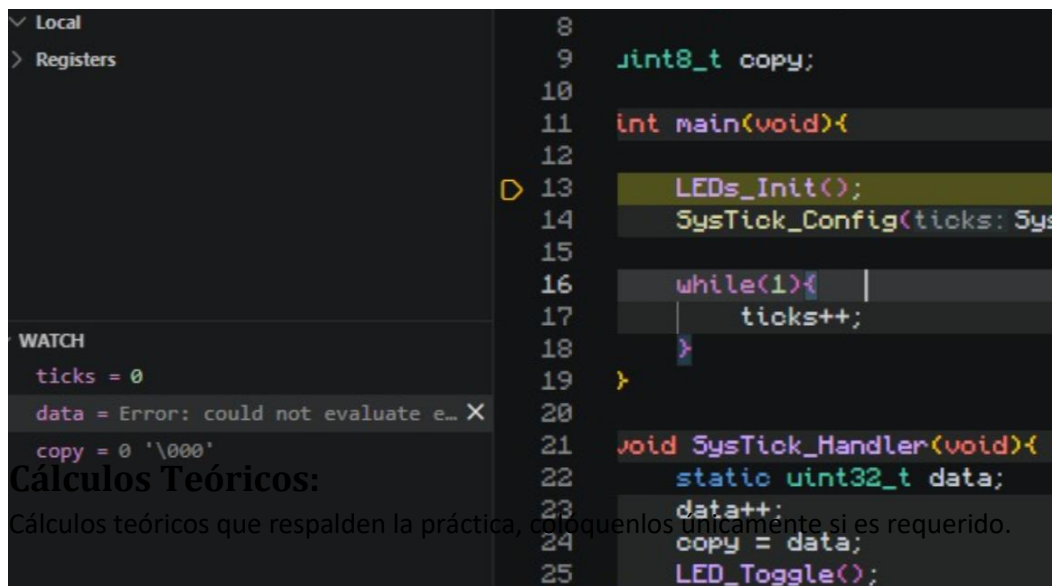
19.



El watch muestra un mensaje de error para data, no es capaz leer su contenido.

Esto ocurre debido a que data esta declarada con el calificador static dentro de la función de la rutina de ejecución, por lo que el depurador no es capaz de evaluar la variable local a no ser que el Program Counter se encuentre detenido dentro de esa función en específico, pero esto no ocurre debido a que al detener la ejecución esta se detiene dentro del bucle principal de while(1) dentro de main().

20.



The screenshot shows a debugger interface with a code editor on the right and a 'WATCH' panel on the left. The code editor displays the following C code:

```
8
9  uint8_t copy;
10
11  int main(void){
12
13      LEDs_Init();
14      SysTick_Config(ticks: Sys
15
16      while(1){
17          ticks++;
18      }
19  }
20
21  void SysTick_Handler(void){
22      static uint32_t data;
23      data++;
24      copy = data;
25      LED_Toggle();
```

The 'WATCH' panel on the left shows the following variables:

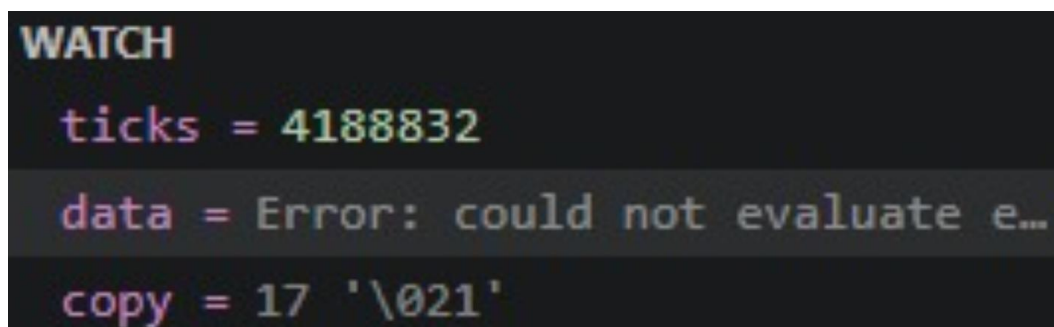
- ticks = 0
- data = Error: could not evaluate e... X
- copy = 0 '\000'

A yellow cursor is positioned at line 13 of the code.

Antes de la ejecución

Cálculos Teóricos:

Cálculos teóricos que respalden la práctica, coloquenlos únicamente si es requerido.



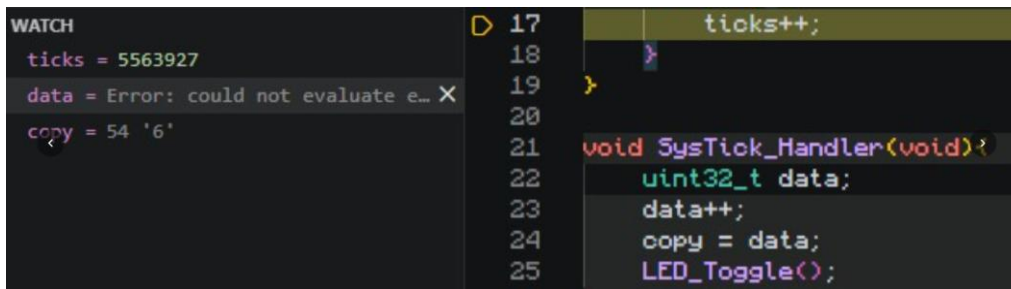
The screenshot shows the 'WATCH' panel of the debugger after execution. The variables are now:

- ticks = 4188832
- data = Error: could not evaluate e...
- copy = 17 '\021'

Después de la ejecución

La variable data sigue mostrando error, pero la variable copy si muestra su valor, el cual se actualiza conforme la ejecución del programa. Esto se debe a que copy es una variable global, por lo que su nombre y su dirección de memoria son visibles para todo el programa y para el depurador en todo momento. Las variables globales se almacenan en una dirección fija de memoria durante toda la ejecución de la aplicación, por esto el depurador es capaz de leer y evaluar el valor de la variable copy sin importar en que punto del código se encuentre.

21.

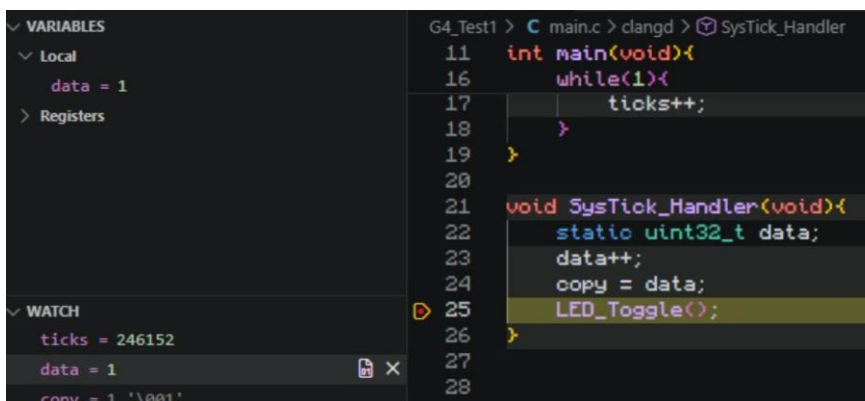


```
WATCH
ticks = 5563927
data = Error: could not evaluate e... X
copy = 54 '6'

17      ticks++;
18  }
19
20
21  void SysTick_Handler(void){
22      uint32_t data;
23      data++;
24      copy = data;
25      LED_Toggle();
```

Lo que ocurre es que data deja de acumular el conteo en las llamadas a la interrupción, por lo que toma datos aleatorios cada vez que entra a SysTick_Handler, por lo cual el valor de copy es inestable. Esto debido a que sin static, data entra al Stack únicamente durante la ejecución de la función y se destruye al salir de ella.

23.



```
VARIABLES
Local
data = 1
Registers

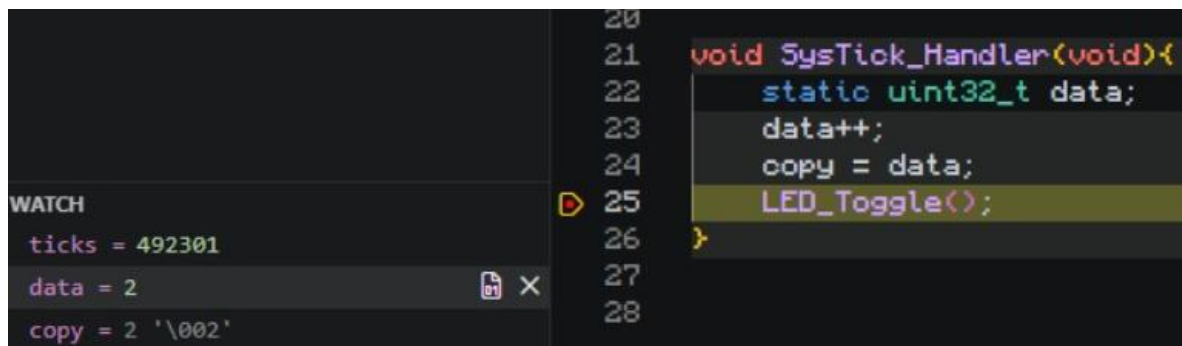
WATCH
ticks = 246152
data = 1
copy = 1 '\001'

G4_Test1 > C main.c > clangd > SysTick_Handler
11  int main(void){
16      while(1){
17          ticks++;
18      }
19
20
21  void SysTick_Handler(void){
22      static uint32_t data;
23      data++;
24      copy = data;
25      LED_Toggle();
26  }
27
28
```

La ejecución del programa se detiene inmediatamente en la línea de LED_Toggle(), frenándose automáticamente cuando el temporizador SysTick genera una interrupción y entra a la función SysTick_Handler().

En este caso, data pasa a ser legible para el depurador porque la ejecución se detuvo dentro de su ámbito local.

24.



```
20
21 void SysTick_Handler(void){
22     static uint32_t data;
23     data++;
24     copy = data;
25     LED_Toggle();
26 }
27
28
```

WATCH

ticks = 492301

data = 2

copy = 2 '\002'

El programa se reanuda, pero se vuelve a detener en la misma línea cuando SysTick vuelve a llamar a la interrupción, en este caso, como data es static, retiene su estado, por lo que data++ incrementa el contador anterior.

25.

No, la aplicación no hace un buen uso de las variables, esto debido a que ticks y data se incrementan sin parar con data++ y ticks++, pero nunca se leen para tomar una decisión lógica ni se asignan a un registro para algún efecto secundario, por lo que se vuelve código redundante.

Para solucionar esto, se debe agregar el calificador volatile para indicar al compilador que el valor de la variable puede cambiar en cualquier momento y que no debe optimizarse, forzando su escritura y lectura en la memoria RAM.

```
7 volatile uint64_t ticks;
8
9 volatile uint8_t copy;
10
11 int main(void){
12     LEDs_Init();
13     SysTick_Config(ticks: SystemCoreClock/FREQ);
14
15     while(1){
16         ticks++;
17     }
18 }
19
20
21 void SysTick_Handler(void){
22     static volatile uint32_t data;
23     data++;
24     copy = data;
25     LED_Toggle();
26 }
```

Conclusiones:

Esta practica es una buena introducción a como trabajar con la tarjeta y a comprender lo más básico de lo que es capaz de hacer, aun así muestra características importantes y detalles los cuales hay que tomar en cuenta, el principal siendo que al trabajar con la tarjeta los recursos son muchísimo más limitados, por lo que es necesario ser consciente de que variables declaramos y como las declaramos dependiendo de para que se van a utilizar, también muestra que la ubicación de las variables en memoria es importante para poder analizarlas correctamente, ya que hay variables que nos interesa que solo vivan en una función mientras que hay otras variables las cuales queremos que se mantengan accesibles durante toda la ejecución del programa. Los breakpoints son una herramienta muy poderosa la cual permite detener la ejecución del programa en un punto de interés cuando se quiere analizar algo específico en algún momento de este.

Bibliografía:

- [1] STMicroelectronics, "STM32G4 Series Microcontrollers," STMicroelectronics Web, 2024.
<https://www.st.com/en/microcontrollers-microprocessors/stm32g4-series.html>

- [2] STMicroelectronics, "STM32 Nucleo Boards," STMicroelectronics Web, 2024.
<https://www.st.com/en/evaluation-tools/stm32-nucleo-boards.html>